

Day 1 Theory: How the Web Works, and Your First Web Page

Workshop: Web Technology, Hands-on Full-Stack Workshop

Audience: BCA Second Year, Mandsaur University

Time for this day: 2 hours in class, plus about 20 minutes of reading before class

How to use this document

Read this document once before class if you can. You do not need to understand everything. Write down the parts that are not clear and bring your questions to class. In class we will build a real web page, and the ideas here will start to make sense when you see them working.

Words in **bold** are new words. They are also collected in the glossary at the end.

1. What we will build in five days

During this workshop you will build one small app called the **Notes Board**. It is a simple notes app. You can write a note, see all your notes, and delete a note. It is small, but it uses every major part of a real web application.

Day	What you learn	What your Notes Board can do at the end of the day
1	How the web works, HTML and CSS	Shows a styled page with sample notes and an About page
2	Tools, the terminal, JavaScript	Adds and deletes notes in the browser
3	A server with Node.js and Express	Keeps notes on a server, adding notes needs a password
4	A database and Git	Notes are saved in a database and kept after restart
5	Hosting	Your app runs on the internet with a public link

By Saturday you will have a project you can show to friends, teachers and future employers.

2. What is the web?

The **internet** is a very large network of computers that are connected to each other. The **web** (World Wide Web) is one service that runs on the internet. It is the collection of websites and web pages that you open with a **browser**.

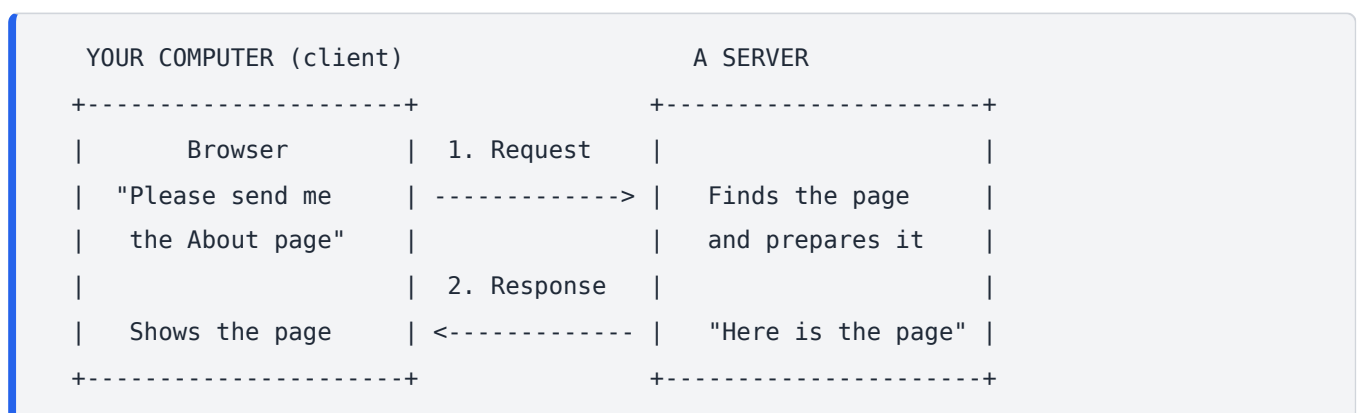
Other services also use the internet. Email, video calls and online games are examples. They are not "the web", but they use the same internet.

The main characters

- **Browser:** A program on your computer that opens web pages. Chrome, Edge and Firefox are browsers.
- **Web page:** One document that a browser can show, for example a page of news.
- **Website:** A group of web pages that belong together.
- **Server:** A computer that is always on and waiting to answer requests. It stores the files of a website and sends them when someone asks.
- **Client:** The computer or phone that asks for a page. Your browser is the client.
- **Address (URL):** The text you type in the browser to find a page, such as `https://example.com/about`.

3. What happens when you open a web page?

When you type an address and press Enter, a conversation starts between your browser and a server.



1. The browser sends a **request**: "Please give me this page."
2. The server finds the page and sends back a **response**: "Here it is."
3. The browser reads the response and shows the page on your screen.

This request and response pattern is the most important idea of the web. Everything we build in this workshop is built on top of it.

The rules for this conversation are called **HTTP**. When you see `https://` at the start of an address, the conversation is also **encrypted**. That means other people on the network cannot read it easily.

4. Frontend, backend and database

A web application has three main parts. The easiest way to remember them is a restaurant.

Restaurant	Web application	Job
Dining room: tables, menu, what the guest sees	Frontend	What the user sees and clicks, inside the browser
Kitchen: cooks follow rules and prepare food	Backend	The logic that runs on the server
Storeroom: ingredients are kept safely	Database	Where data is stored for a long time

```
Guest  -> Dining room  -> Kitchen  -> Storeroom
User   -> Frontend      -> Backend   -> Database
```

Frontend

The **frontend** is everything that runs inside the user's browser. It is the buttons, the text, the colours and the forms. It is built with three technologies, which you will learn in this workshop:

- **HTML** for the structure and content.
- **CSS** for the look and style.
- **JavaScript** for the behaviour (what happens when you click).

Backend

The **backend** is the part that runs on the server. The user never sees it directly. It receives requests from the frontend, applies the rules (for example "only allow people with the right password"), talks to the database, and sends back an answer. In this workshop we use **JavaScript with Node.js and Express** for the backend, so you only need to learn one programming language for both sides.

Database

A **database** is an organised place to store data so that it is not lost when you close the program. If you only keep data in the browser's memory, it disappears when you refresh the page. A database keeps it safe. We will use **SQLite** in this workshop. It is a small database that stores everything in one file. Some students may also try **PostgreSQL**, a bigger database used by many companies.

Full-stack

A **full-stack developer** can work on the frontend, the backend and the database. This workshop gives you a first taste of all three.

5. HTML, CSS and JavaScript: the three layers

Think of a house.

Part of the house	Web technology	What it does
Walls, doors, rooms	HTML	Gives the page its structure and content
Paint, furniture, decoration	CSS	Makes the page look good
Electricity, switches, doors that open	JavaScript	Makes the page do things

Today we learn HTML and CSS. JavaScript starts on Day 2.

6. Tools we use

Text editor and IDE

Code is just text. You can write it in any text editor, even Notepad. But programmers use better tools.

- A **code editor** is a text editor made for code. It colours the words, completes them for you, and warns you about mistakes.
- An **IDE** (Integrated Development Environment) is a bigger tool that also helps you run, test and fix your program. It brings many tools into one window.

Visual Studio Code (VS Code) is the tool we use. It is free, it is already installed on your computer, and it works like a light IDE. Many professional developers use it.

Useful VS Code parts:

- **Explorer** (left side): shows your folders and files.
- **Editor** (centre): where you write code.
- **Terminal** (bottom): where you type commands. We use it from Day 2.
- **Extensions** (left side): add-ons that give VS Code new powers.

The command line (CLI)

A **CLI** (Command Line Interface) lets you control your computer by typing commands instead of clicking. The window where you type is called the **terminal**. It looks plain, but it is fast and powerful. We start using it on Day 2, and we will explain every command we use.

Browser developer tools

Every modern browser has **developer tools**. Press **F12** to open them. They let you look at the HTML of a page, test CSS changes, and see errors. They are a developer's best friend.

7. HTML: the structure of a page

HTML stands for **HyperText Markup Language**. It is not a programming language. It is a way to *mark up* text so the browser knows what each part is: a heading, a paragraph, a link, a button.

Tags and elements

HTML uses **tags** written inside angle brackets. Most tags come in pairs: an opening tag and a closing tag. The closing tag has a slash.

```
<p>This is a paragraph.</p>
```

- `<p>` is the opening tag.
- `</p>` is the closing tag.
- The whole thing, from the opening tag to the closing tag, is called an **element**.

Some elements can sit inside other elements. This is called **nesting**:

```
<section>
  <h2>My title</h2>
  <p>My text</p>
</section>
```

Attributes

An **attribute** gives extra information to a tag. It is written inside the opening tag as `name="value"` .

```
<a href="https://example.com">Visit the example website</a>
```

Here `href` is the attribute name and the address is its value.

The basic structure of every HTML page

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>My Notes Board</title>
  </head>
  <body>
    <h1>Hello</h1>
  </body>
</html>
```

Part	Meaning
<code><!DOCTYPE html></code>	Tells the browser this is a modern HTML page
<code><html lang="en"></code>	The root of the page, and the language is English
<code><head></code>	Information about the page that is not shown on the page itself
<code><meta charset="UTF-8"></code>	Lets the page show all kinds of letters and symbols correctly
<code><title></code>	The text on the browser tab
<code><body></code>	Everything the user sees on the page

Common tags

Tag	Use
<code><h1></code> to <code><h6></code>	Headings. <code>h1</code> is the biggest and most important
<code><p></code>	A paragraph of text
<code></code>	A link to another page
<code></code>	An image
<code></code> and <code></code>	A bullet list and its items
<code><div></code>	A plain box used to group other elements
<code><header></code> , <code><main></code> , <code><section></code> , <code><article></code> , <code><footer></code>	Boxes with a meaning, which make the page easier to understand
<code><nav></code>	A group of navigation links
<code><form></code>	A form where a user can type and send information
<code><label></code>	The name of an input field
<code><input></code>	A single-line field
<code><textarea></code>	A multi-line text box
<code><button></code>	A button

Classes and ids

To style or find an element later, we give it a name:

- `class="note"` can be used on many elements.
- `id="note-form"` must be used on only one element in the page.

8. CSS: the style of a page

CSS stands for **Cascading Style Sheets**. It tells the browser how things should look.

A CSS rule

```
h1 {  
  color: blue;  
  font-size: 32px;  
}
```

- `h1` is the **selector**. It says which elements to style.
- `color` and `font-size` are **properties**.
- `blue` and `32px` are **values**.
- Each line ends with a semicolon `;`.

Selectors you will use

Selector	Example	Meaning
Tag	<code>p { ... }</code>	All paragraphs
Class	<code>.note { ... }</code>	All elements with <code>class="note"</code>
Id	<code>#note-form { ... }</code>	The one element with <code>id="note-form"</code>

Three ways to add CSS

1. **Inline:** inside a tag, as `style="..."`. Quick, but hard to manage.
2. **Internal:** inside a `<style>` tag in the page.
3. **External:** in a separate `.css` file, linked with `<link rel="stylesheet" href="css/style.css">`. This is the best way and the one we use.

The box model

Every element is a rectangular box. The box has four layers:



Colours and units

- Colours can be names (`red`), hex codes (`#2563eb`), or other forms.
- `px` means pixels, a fixed size. `rem` means a size relative to the base text size. `%` is a share of the parent box.

CSS variables

We store colours in one place so that we can change the whole look quickly:

```
:root {
  --accent: #2563eb;
}

button {
  background: var(--accent);
}
```

9. Files and folders

A website is a folder of files. The names and the places of the files matter.

```
notes-board/  
  index.html      the main page (the home page)  
  about.html      the About page  
  css/  
    style.css      the style rules
```

- `index.html` is the usual name for the home page.
- A **path** tells the browser where a file is. `css/style.css` means "go into the folder `css` and take the file `style.css`".
- Use lowercase names, no spaces, and no special letters in file names.
- Windows can hide the file extension (the `.html` part). We will turn extensions on in the lab.

10. A first look at forms

A form lets the user type something and send it. Our Notes Board has a form with a title, a note, and a button. Today the form does not save anything. The page only reloads when you press the button, and this is normal. On Day 2 we use JavaScript to make it work, and on Day 3 we send it to a server.

11. Quick revision

1. A browser sends a **request** and a server sends a **response**.
2. **Frontend** is what the user sees. **Backend** is the logic on the server. **Database** stores data.
3. **HTML** is structure, **CSS** is style, **JavaScript** is behaviour.
4. **VS Code** is the tool where we write code. The **terminal** is where we type commands.
5. A web page is files in folders, and a **path** tells where each file is.

12. Homework for Day 2

Day 2 starts with installing software. To save time, do these two things before class on Day 2.

Task A: Collect the installers

Your instructor will tell you where to find the workshop installer folder. Copy these files into a folder named `Workshop-Installers` on your computer or on a USB drive:

1. The **Node.js 22 LTS** installer for Windows, 64-bit (a file ending in `.msi`).
2. The **Git for Windows** installer, 64-bit (a file ending in `.exe`).

Please take the files only from the folder your instructor points to. Other versions, such as "Current" releases or 32-bit files, can cause problems later.

If you want to download them yourself, ask your instructor for the exact download pages first.

Task B: Create a GitHub account

GitHub is a website where developers keep and share their code. You will need it on Day 4 and Day 5.

1. Go to the GitHub website and choose **Sign up**.
2. Use an email address that you can open. You will get a code or link to verify it.
3. Pick a simple, professional username, for example your name. Employers may see it one day.
4. Keep your password safe. Do not share it with anyone.

If you cannot do this today, tell your instructor on Day 2. Do not worry. We will find a way.

Task C: Write your questions

Write down at least two things that were not clear. Bring them to class.

Glossary

Word	Meaning
Attribute	Extra information inside an HTML tag, such as <code>href</code>
Backend	The part of an app that runs on the server
Browser	A program that opens web pages
CLI	Command Line Interface: controlling the computer by typing commands
Client	The computer or phone that asks a server for something
CSS	The language that styles a web page
Database	An organised store for data that stays saved
Developer tools	A set of tools inside the browser for checking a page (F12)
Element	An opening tag, its content and its closing tag
Frontend	The part of an app that runs in the browser
Full-stack	Working on the frontend, backend and database
HTML	The language that gives a web page its structure
HTTP	The rules that a browser and a server use to talk
IDE	A tool that brings many programming tools into one window
Internet	The worldwide network of connected computers
JavaScript	The programming language that makes pages interactive
Path	The location of a file, such as <code>css/style.css</code>
Request	A message from the browser asking for something
Response	The answer that a server sends back
Server	A computer that waits for requests and sends responses
Terminal	The window where you type commands
URL	The address of a web page
Web	The collection of websites that you open in a browser